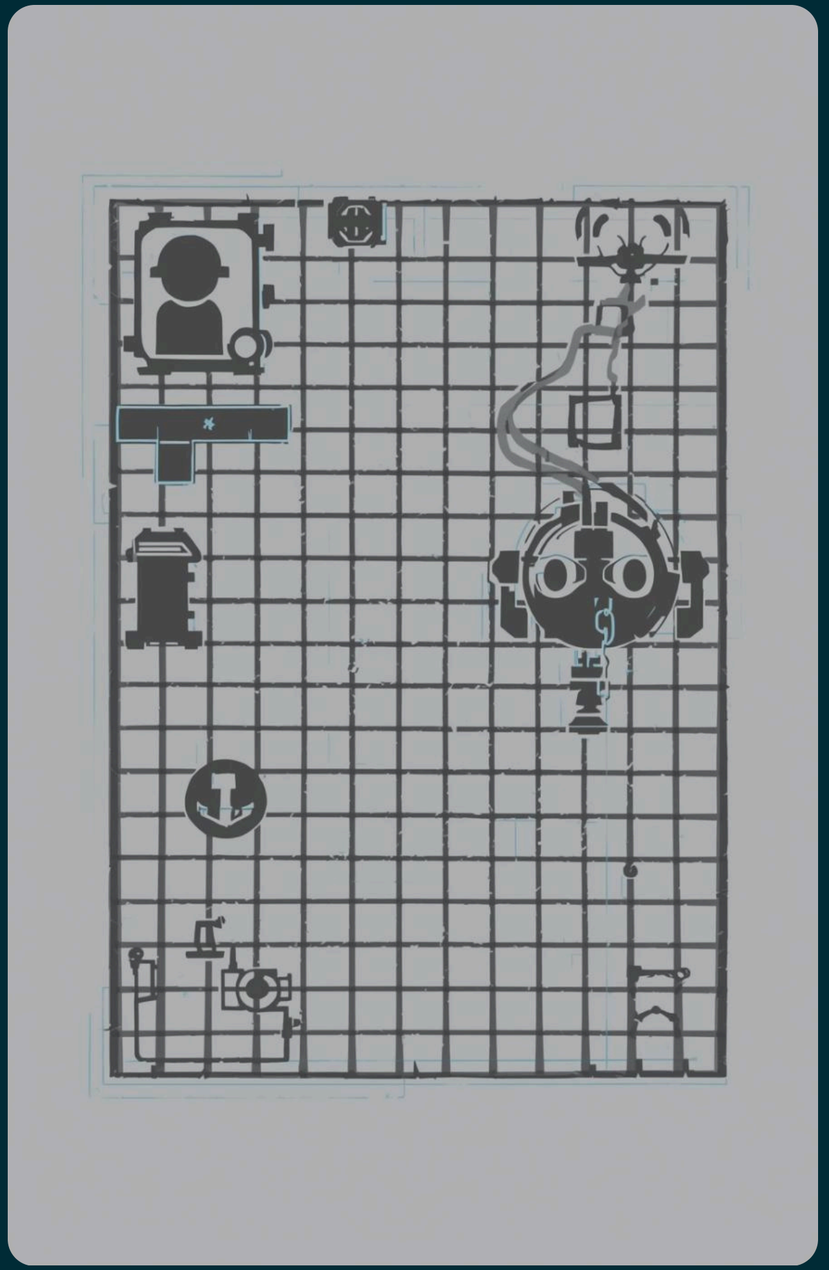
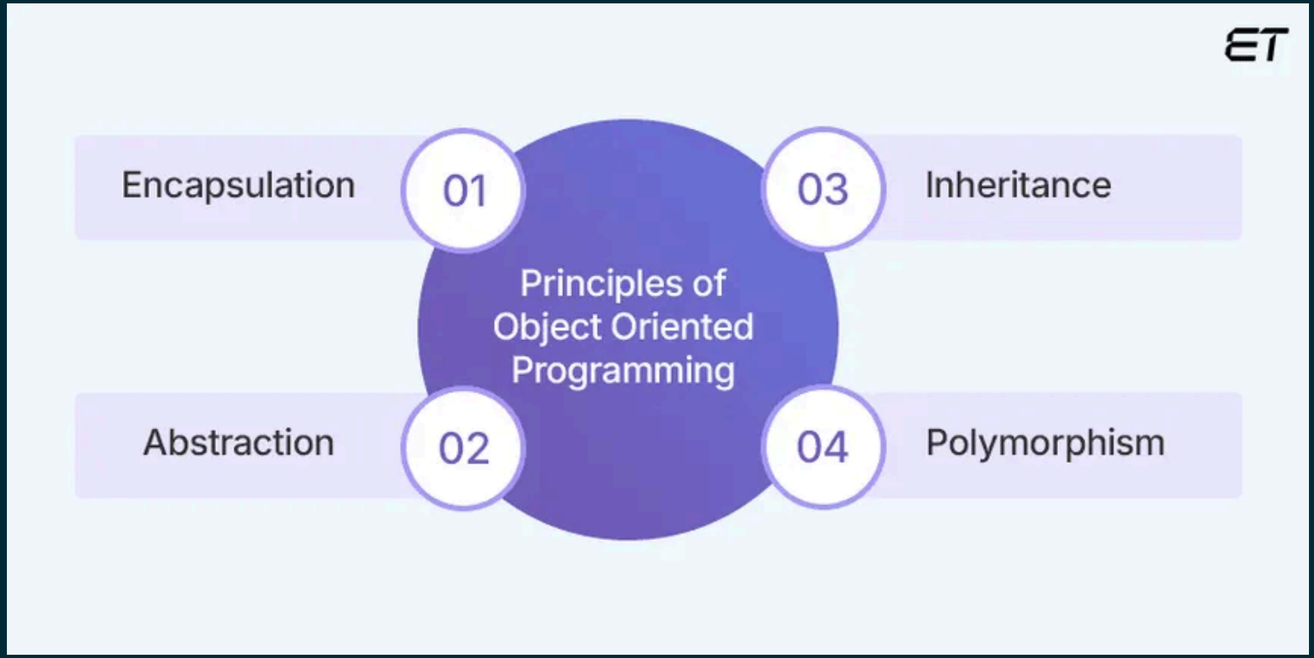
Projet Java : Jeu Chasseur / Bonhomme

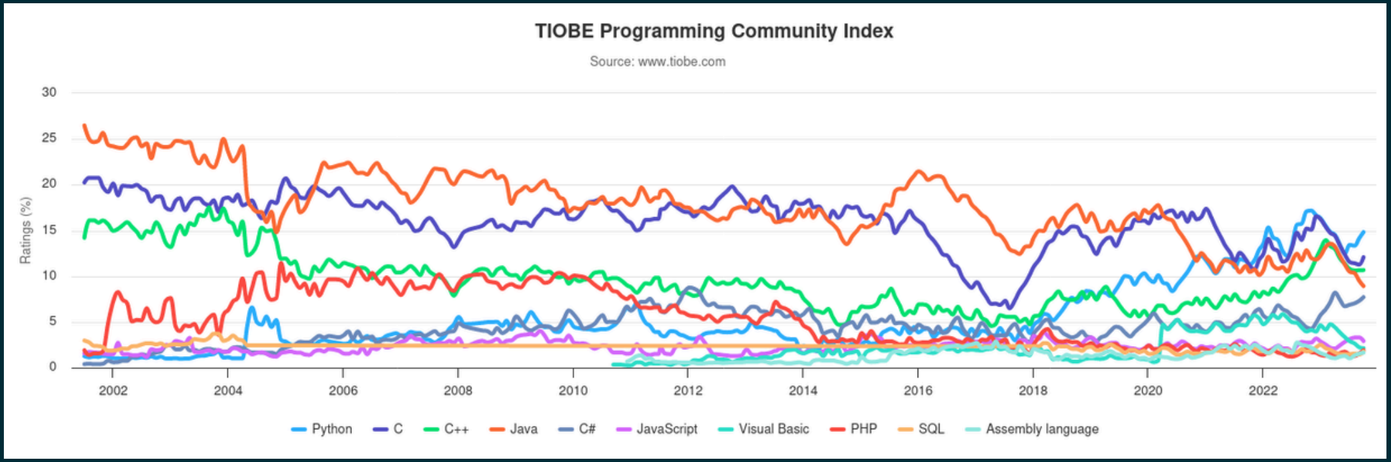
https://github.com/Romain-Jaffuel/Hunter-and-Man-Game/

Présenté par
Romain Jaffuel & Florian Grolleau



Objectif du projet





L'objectif principal est de **développer une version fonctionnelle** du jeu en Java, en utilisant la programmation orientée objet

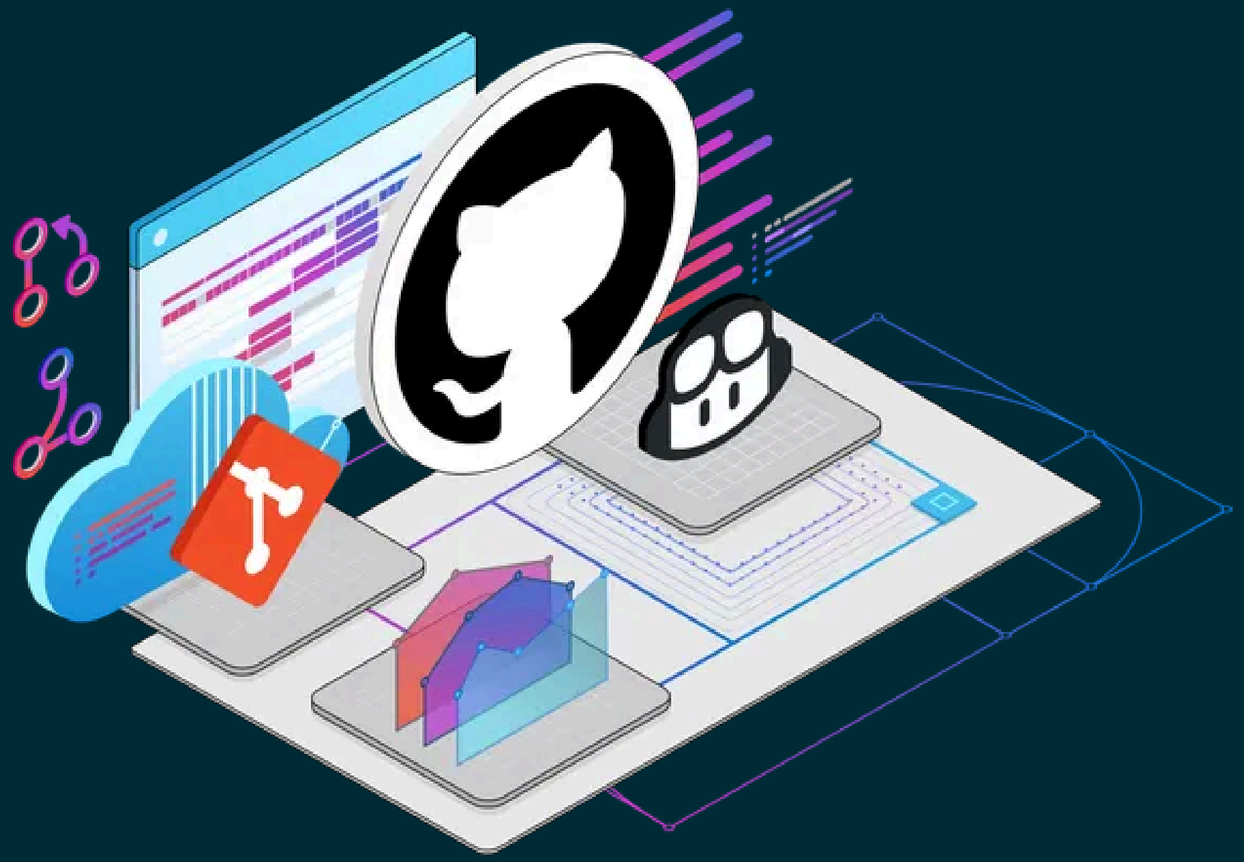
Organisation & logique de travail

Construire l’architecture

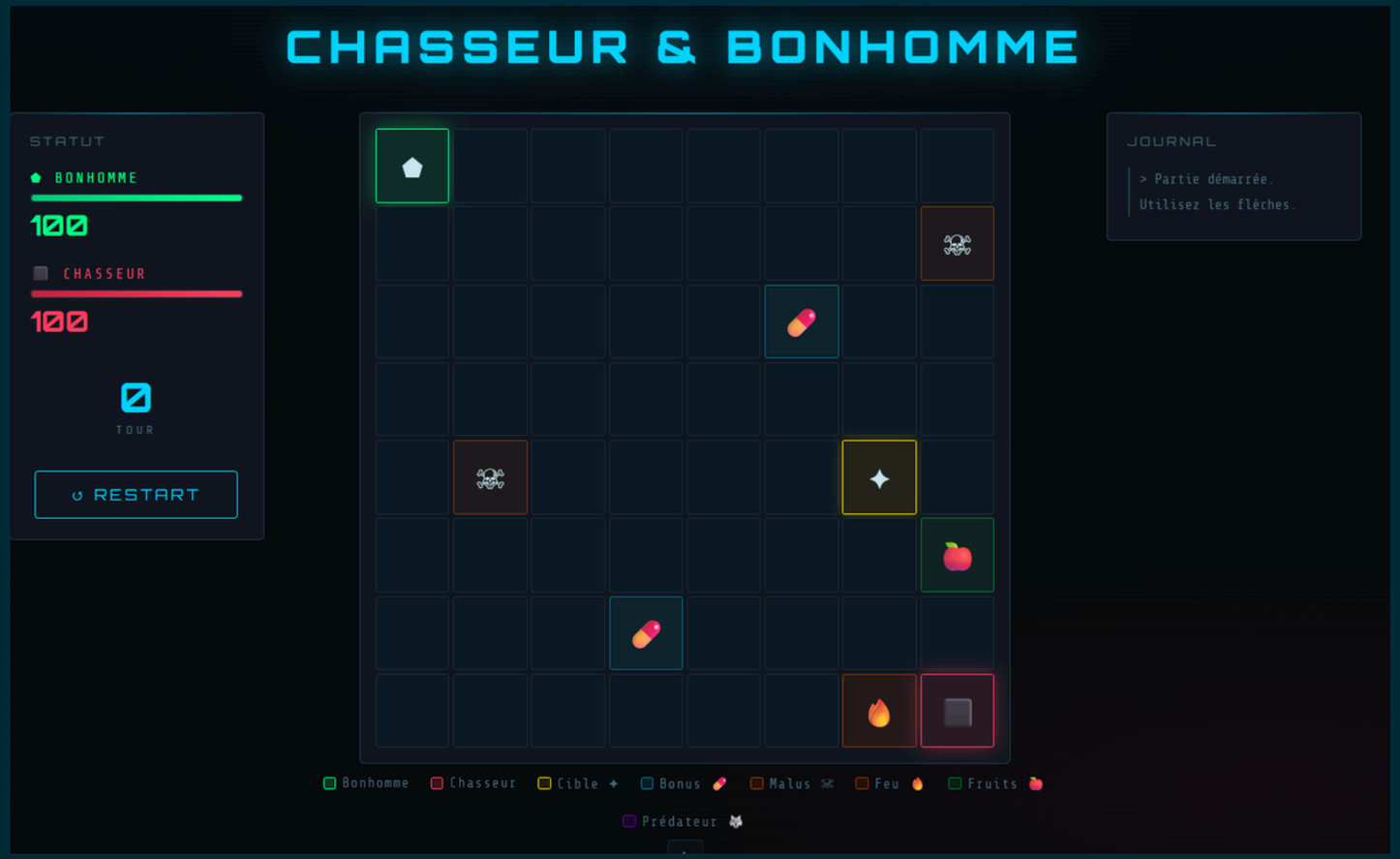
a. UML
b. Classes
c. Méthodes

Loop de travail :

1. Rencontrer des problèmes
2. Proposer des solutions
3. Adopter une solution
4. Coder étape par étape

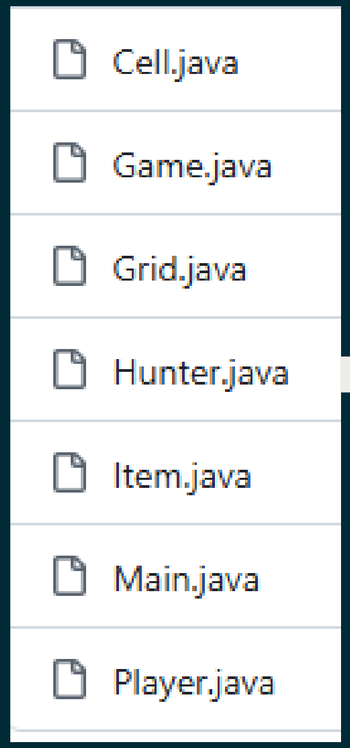


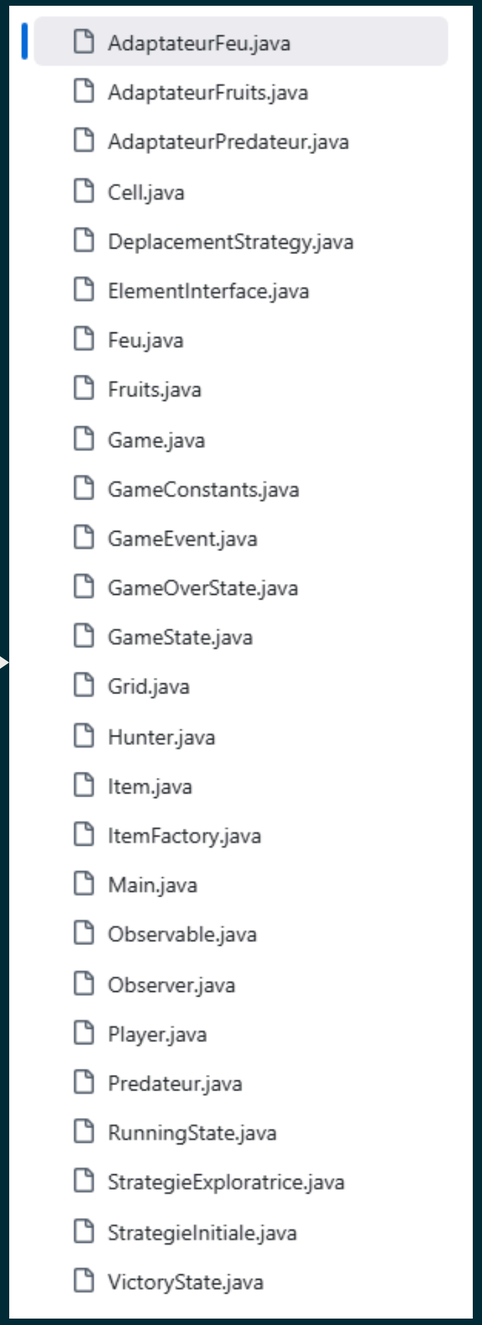
Principe du jeu : Grille et énergie



Ce jeu se déroule sur une **grille dynamique** où le bonhomme interagit avec un chasseur automatique, utilisant une ressource d'énergie pour effectuer des déplacements stratégiques tout au long de la partie.

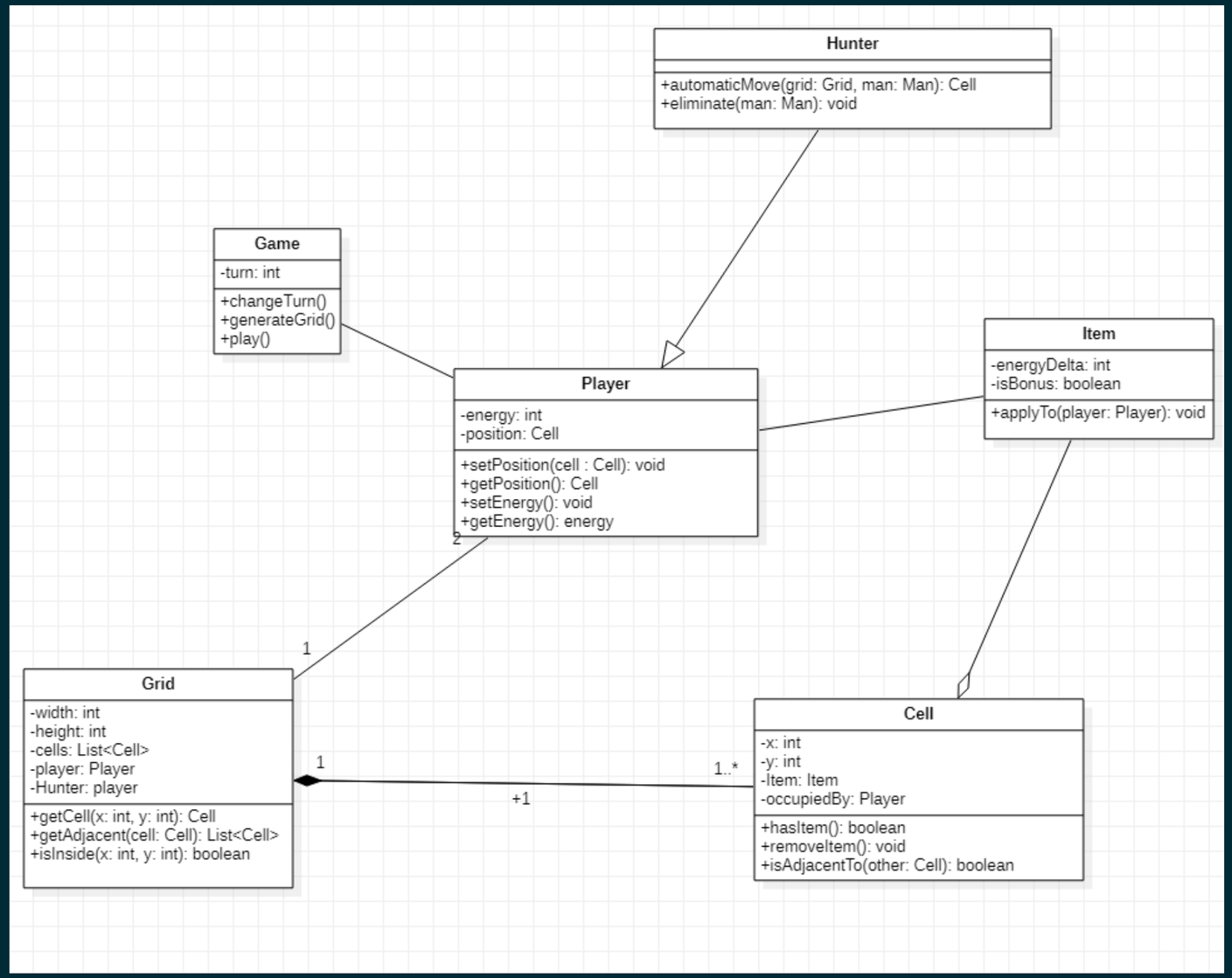
Architecture du programme





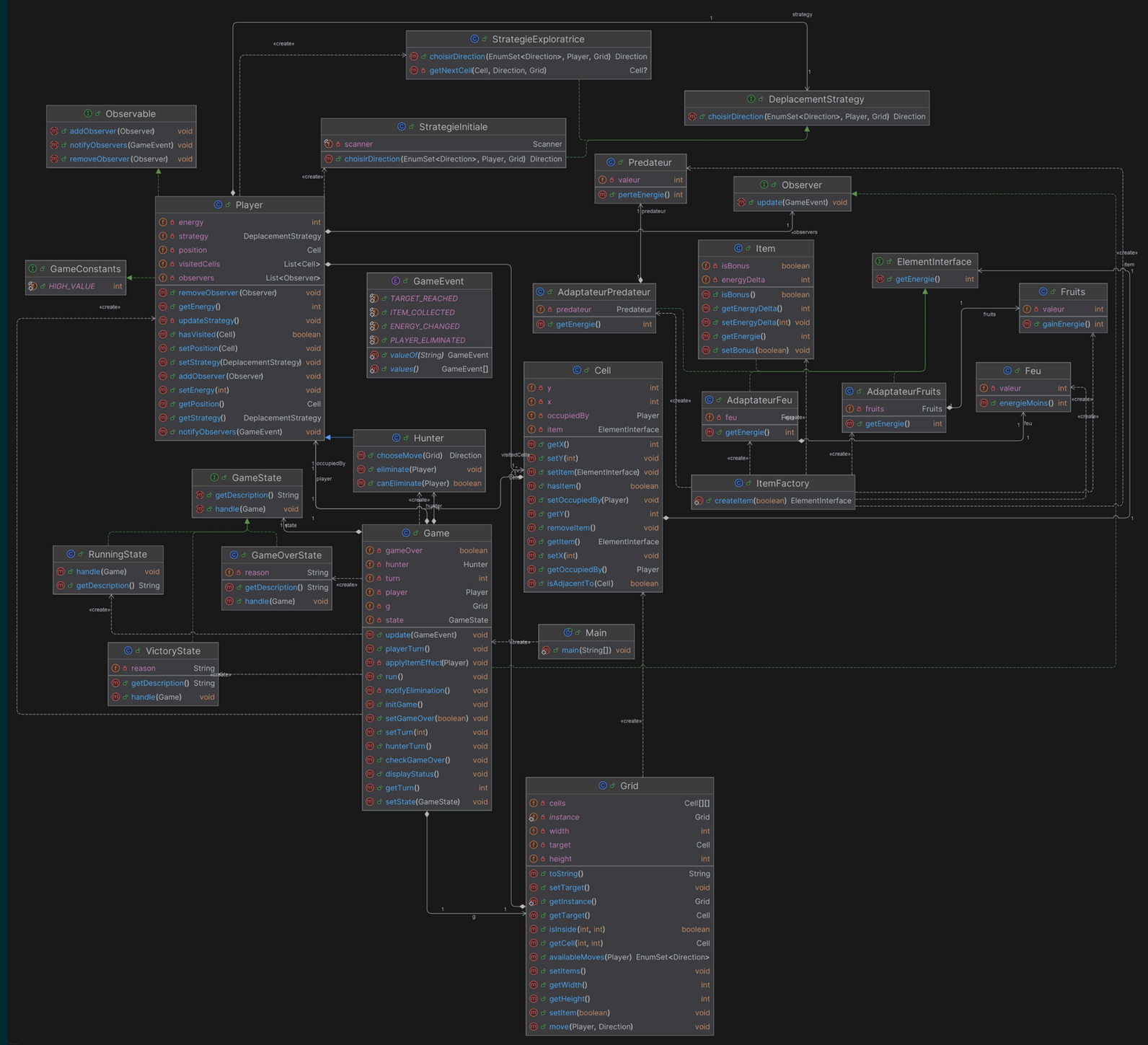
Cette section présente les classes principales du jeu, notamment **Game**, **Grid**, **Cell**, **Player**, **Hunter**, et **Element**, soulignant leur rôle et leur interaction dans la structure du programme.

Organisation des classes



Ce diagramme UML illustre les relations entre les principales classes du projet. Les classes principales incluent Game, Grid, Cell, Player, Hunter et Element, démontrant l'interaction entre les objets du jeu.

Organisation des classes



```
P . . . . . . . .
. . . . I . . . .
I . I I . . . . .
. . . . . . . . .
. . . . . . . . .
. . . . . . . . I
. . T . . . I . .
. . . . . . . . H

Turn : 0
Hunter's energy : 100
Player's energy : 100
Available moves: [S, E]
Choose direction:
```

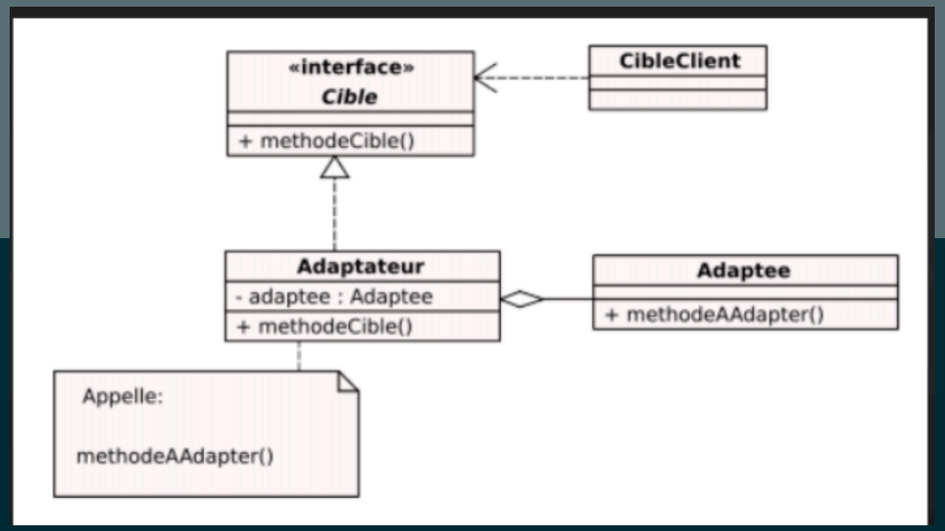
Difficultés techniques

Qu'une action soit bien

1. Complète → Ex : Bouger le *Player*, le retirer de la *Cell* précédente et que la *Grid* le prenne en compte
2. Visuel → Gestion du *toString()*

Les *Malus* qui ont forcé à recoder la logique d'apparition des *Items*

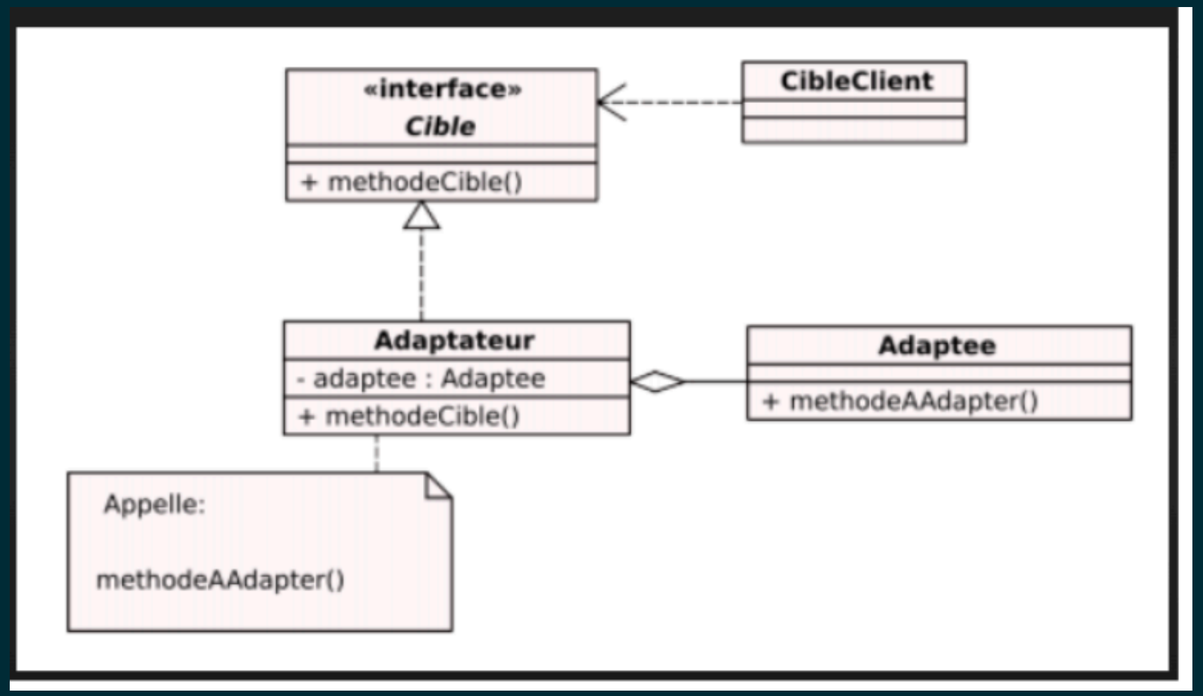
Q4-Q5 : Adaptateur



- Fermeture à la modification
- Ouverture à l'extension

```
3   public class AdaptateurFruits implements ElementInterface {  1 usage
4       private Fruits fruits;  2 usages
5       public AdaptateurFruits(Fruits fruits) { this.fruits = fruits; }  1 usage
6
7       @Override  2 usages
8       public int getEnergie() {
9           return fruits.gainEnergie(); // adapte gainEnergie() → getEnergie()
10      }
11  }
```

Q6-Q7 Stratégies, encapsulation des algorithmes interchangeables



<110 : il demande au joueur (qui correspond à la stratégie initiale)
>110 il choisit automatiquement une case non visitée

```java
package classique;

public interface GameConstants {
    int HIGH_VALUE = 110;
}
```

```java
package classique;

import java.util.EnumSet;

public interface DeplacementStrategy {
    Grid.Direction choisirDirection(EnumSet<Grid.Direction> moves, Player player, Grid grid);
}
```

Q6-Q7 Stratégies, encapsulation des algorithmes interchangeables

```java
            System.out.print("Choose direction: ");
            String choice = scanner.nextLine().trim().toUpperCase();
            try {
                Grid.Direction tmp = Grid.Direction.valueOf(choice);
                if (moves.contains(tmp)) {
                    chosen = tmp;
                } else {
                    System.out.println("Direction not allowed from this position.");
                }
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid input. Use N, S, E or W.");
            }
        }
        return chosen;
```

```java
public class Player implements GameConstants {
    private DeplacementStrategy strategy;
    private List<Cell> visitedCells = new ArrayList<>(); // ← le "papier"

    public Player() {
        strategy = new StrategieInitiale(); // ← stratégie par défaut
    }
```

```java
        for (Grid.Direction dir : moves) {
            Cell next = getNextCell(current, dir, grid);
            if (next != null && !player.hasVisited(next)) {
                System.out.println("Strategie exploratrice : direction " + dir);
                return dir;
            }
        }
        // Toutes adjacentes visitées → première direction disponible
        System.out.println("Toutes les cases visitées, direction par défaut.");
```

```java
private void updateStrategy() {
    if (energy > GameConstants.HIGH_VALUE) {
        strategy = new StrategieExploratrice();
    } else {
        strategy = new StrategieInitiale();
    }
}

public boolean hasVisited(Cell cell) {
    return visitedCells.contains(cell);
}
```

Q8 Pattern Observer notification immédiate

```java
package classique;

public enum GameEvent {
    ENERGY_CHANGED,
    PLAYER_ELIMINATED,
    TARGET_REACHED,
    ITEM_COLLECTED
}
```

```java
public void setEnergy(int energy) {
    this.energy = energy;
    updateStrategy();
    notifyObservers(GameEvent.ENERGY_CHANGED);
}
```

```java
package classique;

public interface Observer {
    void update(GameEvent event);
}
```

```java
@Override
public void update(GameEvent event) {
    // ← cette méthode est appelée AUTOMATIQUEMENT par Player/Hunter
    switch (event) {
        case ENERGY_CHANGED:
            checkGameOver(); // ← plus besoin d'appel manuel dans run()
            break;
        case PLAYER_ELIMINATED:
            gameOver = true;
            System.out.println("Game Over ! Hunter eliminated the player.");
            break;
        case TARGET_REACHED:
            gameOver = true;
            System.out.println("Victory ! Target reached.");
            break;
        case ITEM_COLLECTED:
            System.out.println("Item collected.");
            break;
    }
}
```

```java
public interface Observable {
    void addObserver(Observer o);       // ← "abonne-toi à moi"
    void removeObserver(Observer o);    // ← "désabonne-toi"
    void notifyObservers(GameEvent event); // ← "envoie la notification à tous"
}
```

Q9 Addition d'autres patterns

```java
public static Grid getInstance() { // ← seul point d'accès
    if (instance == null) {
        instance = new Grid(); // ← créée une seule fois
    }
    return instance;
}

public static void resetInstance() {
    instance = null; // ← pour les restarts
}
```

```java
private Grid g = Grid.getInstance(); // ← plus "new Grid()"
```

```java
        if (isBonus) {
            int choice = rand.nextInt(2);
            if (choice == 0) {
                Item item = new Item();
                item.setEnergyDelta(rand.nextInt(30) + 1);
                item.setBonus(true);
                return item;
            } else {
                return new AdaptateurFruits(new Fruits(rand.nextInt(30) + 1));
            }
        } else {
            int choice = rand.nextInt(3);
            switch (choice) {
                case 0:
                    Item item = new Item();
                    item.setEnergyDelta(-(rand.nextInt(20) + 1));
                    item.setBonus(false);
                    return item;
                case 1: return new AdaptateurFeu(new Feu(rand.nextInt(20) + 1));
                default: return new Adap urPredateur(new Predateur(rand.nextInt(20)
            }
```

```java
Item item = new Item();
item.setEnergyDelta(rand.nextInt(30) + 1);
item.setBonus(true);
cell.setItem(item);
```

```java
cell.setItem(ItemFactory.createItem(isBonus));
```

Q9 Pattern etat du jeu

```java
public class RunningState implements GameState {
    @Override
    public void handle(Game game) { } // ← ne fait rien, le jeu continue
    @Override
    public String getDescription() { return "Game is running"; }
}

public class GameOverState implements GameState {
    private String reason;
    public GameOverState(String reason) { this.reason = reason; }

    @Override
    public void handle(Game game) {
        game.setGameOver(true); // ← arrête le jeu
    }
    @Override
    public String getDescription() { return "Game Over : " + reason; }
}

public class VictoryState implements GameState {
    private String reason;
    public VictoryState(String reason) { this.reason = reason; }
```